

Intune-JIRA Cloud Integration Guide

Hardware and Software Inventory Management

Document Version: 1.0

Last Updated: February 2026

Migration Context: JIRA DC (AD on-prem) → JIRA Cloud with Intune-managed laptops

Table of Contents

1. [Executive Summary](#)
 2. [Available Integration Options](#)
 3. [Azure Entra ID Integration](#)
 4. [Recommended Integration Approaches](#)
 5. [JIRA DC to Cloud Assets Migration](#)
 6. [Implementation Steps](#)
 7. [Known Limitations and Challenges](#)
 8. [Vendor Recommendations](#)
 9. [Required Infrastructure and Prerequisites](#)
 10. [Post-Migration Considerations](#)
-

Executive Summary

This guide outlines integration strategies for connecting Microsoft Intune inventory data with JIRA Cloud Assets (formerly Insight) during a migration from JIRA Data Center to JIRA Cloud. The primary objective is to maintain hardware and software inventory visibility in JIRA Service Management for Intune-managed laptops.

Key Findings:

- **No native Atlassian integration exists** between Intune and JIRA Cloud Assets as of February 2026
 - **Multiple third-party solutions** are available via Atlassian Marketplace
 - **Azure Entra ID provides SSO authentication** but does NOT pull inventory data directly
 - **Assets Data Manager** (included with JSM Premium/Enterprise) offers Intune adapter capability
 - **Custom API integration** is possible but requires development effort
-

Available Integration Options

Option 1: Marketplace Apps (Recommended for Most Organizations)

1.1 Intune Connect for JSM Assets (T4S Partners)

Marketplace Link: <https://marketplace.atlassian.com/apps/1231619/intune-connect-for-jsm-assets>

Capabilities:

- Direct integration between Microsoft Intune and JSM CMDB
- Automated asset synchronization (hourly, daily, or weekly)
- Imports devices with real-time status updates
- Future support planned for applications and configurations

Pricing: Paid via Atlassian (commercial license)

Pros:

- Purpose-built for Intune-JIRA integration
- Automated synchronization eliminates manual data entry
- Regular updates and vendor support
- Enhanced reporting and analytics

Cons:

- Additional licensing cost
 - Currently limited to device imports (apps/configs planned for future)
 - Vendor-dependent for updates and support
-

1.2 Microsoft Intune Importer for JSM Assets (Forge-based)

Marketplace Link: <https://marketplace.atlassian.com/apps/1230667/microsoft-intune-importer-for-jsm-assets>

Capabilities:

- Built on Atlassian Forge platform (enterprise-grade security)
- Creates: Managed Devices, Discovered Applications, Manufacturers, Models, Operating Systems, OS Versions, Users
- Quick setup (< 5 minutes claimed)
- Synchronizes data from Microsoft Intune

Pricing: Paid via Atlassian (commercial license)

Pros:

- Forge platform provides enterprise security and scalability
- Comprehensive object type creation
- Quick deployment
- Latest version: 6.3.0 (January 2026)

Cons:

- Additional licensing cost
 - Limited community feedback available
-

1.3 Asset Management for Jira (AMFJ) with Intune Integration

Capabilities:

- Native Intune integration feature
- Syncs devices and users from Intune
- Automatic sync every 6 hours after initial setup
- Customizable synchronization behavior
- Auto-checkout devices to primary users (optional)
- Filter by ownership type (Corporate, Personal, BYOD)

Configuration Options:

- Default asset type assignment
- Default location settings
- Primary user auto-checkout
- Ownership type filtering

Pros:

- Comprehensive sync including user relationships
- Flexible configuration options
- Established integration with proven track record

Cons:

- Separate asset management solution (not native JSM Assets)
 - Additional platform cost
-

Option 2: Assets Data Manager (Native Atlassian Solution)

Availability: Included with JSM Premium and Enterprise plans at no additional cost

Intune Adapter Capabilities:

- API-based connection to Microsoft Intune
- Requires: Tenant-ID, Client-ID, Client Secret
- API calls: Device.Read.All and Users.Read.All
- Configurable API timeout (recommended: 9000ms, minimum: 2000ms)

Data Manager Clients for JSM Assets (Enhanced Version)

- Marketplace app that enhances Data Manager
- Manager enrichment for Intune devices (requires User.Read.All permission)
- Adds manager fields: ManagerId, ManagerDisplayName, ManagerPrincipalName
- Improved file upload mechanism
- Latest version: 2.0.0 (December 2025)

Workflow:

1. Configure Intune adapter in Data Manager
2. Set up authentication with Azure App Registration
3. Schedule automated imports
4. Data cleansing and normalization
5. Create golden data source in Assets

Pros:

- No additional cost for JSM Premium/Enterprise customers
- Native Atlassian solution with ongoing development
- Supports 30+ data source integrations
- Built-in data cleansing and deduplication

- Scheduled automated imports

Cons:

- Currently in production use but newer feature
 - Requires manual adapter configuration
 - Learning curve for Data Manager platform
 - No generic REST API adapter (as of April 2025)
-

Option 3: Third-Party Integration Platforms

3.1 xAssets

Solution Type: Third-party CMDB/Asset Management platform

Capabilities:

- Centralized data repository combining Intune, SCCM, Jamf, Network Discovery
- Direct integration with JIRA Service Management
- Displays Intune asset data directly in JIRA tickets
- Automatic asset population in tickets based on:
 - Person names in ticket text
 - IP addresses
 - Computer names
 - Reporter's laptop/PC
- Provides replacement for deprecated Atlassian Asset Panel (removed January 2025)
- Network discovery for comprehensive hardware/software/patch/security data
- Free tier: up to 100 endpoints (no integrations)
- Paid tier: full integrations with Intune, SCCM, Meraki, MDMs

Pros:

- Comprehensive cross-platform asset management
- Enhanced asset context in JIRA tickets
- Deep discovery capabilities beyond Intune data
- Fills gap left by deprecated Atlassian Asset Panel

Cons:

- Additional platform/licensing cost
 - Introduces new system to manage
 - Data lives in xAssets, not natively in JIRA Assets
-

3.2 Workativ Automation Platform

Solution Type: No-code workflow automation

Capabilities:

- Connects Intune and JIRA Service Management
- Pre-built workflows for device management
- No coding required
- UI-based integration builder

Use Cases:

- Onboarding/offboarding workflows
- Automated ticket creation for device events
- Data synchronization between systems

Pros:

- No-code solution
- Flexible workflow automation
- 100+ cloud app connections

Cons:

- Additional platform cost
 - Workflow-focused rather than direct asset sync
 - May require ongoing workflow maintenance
-

Option 4: Custom REST API Integration

Approach: Build custom integration using:

- Microsoft Graph API (Intune data source)

- JIRA Assets REST API (destination)
- Scheduled synchronization script (PowerShell, Python, or other)

Requirements:

- Azure App Registration with appropriate permissions
- Development resources for script creation
- Hosting for synchronization script (Azure VM, on-prem server, or scheduled cloud function)
- Ongoing maintenance and monitoring

Microsoft Graph API Endpoints:

```
GET /deviceManagement/managedDevices  
GET /users  
GET /deviceManagement/detectedApps
```

JIRA Assets REST API:

```
https://api.atlassian.com/ex/jira/{cloudId}/jsm/assets/workspace/{workspaceId}/v{version}/object
```

Sample Implementation Approaches:

1. **PowerShell Script** (scheduled via Azure Automation or Task Scheduler)
2. **Azure Function** (serverless, event-driven or scheduled)
3. **Python Script** (with libraries: requests, pandas for data transformation)
4. **Integration Server/VM** (dedicated Windows/Linux server running sync service)

Pros:

- No additional licensing cost (beyond hosting)
- Full control over data mapping and transformation
- Customizable sync frequency and filtering
- Can include custom business logic

Cons:

- Requires development expertise
- Ongoing maintenance responsibility
- No vendor support for integration logic
- Must handle API changes and authentication refresh

- Security considerations for credential management
-

Azure Entra ID Integration

Important Clarification: SSO vs. Inventory Data

Azure Entra ID provides:

- Single Sign-On (SSO) authentication via SAML 2.0 or OIDC
- User authentication and authorization
- User provisioning (accounts and groups)

Azure Entra ID does NOT provide:

- Direct access to Intune inventory data
- Hardware/software asset information
- Device management data

SSO Integration (Authentication Only)

JIRA Cloud supports Azure Entra ID for user authentication through:

1. Atlassian Access (Cloud)

- SAML SSO configuration
- User provisioning via SCIM
- Group sync
- Requires separate Atlassian Access license

2. SAML SSO Plugins (Data Center/Server - not applicable for Cloud migration)

- Microsoft JIRA SAML SSO plugin
- Third-party SAML providers

3. Configuration Steps for JIRA Cloud:

- Configure Atlassian Access in your organization
- Set up Azure Entra ID as Identity Provider
- Configure SAML settings
- Map user attributes
- Test SSO authentication

Important: SSO integration is separate from asset data integration. You will need:

- **Azure Entra ID** for user authentication
 - **Separate integration solution** (Options 1-4 above) for Intune inventory data
-

Recommended Integration Approaches

Scenario 1: Standard Organization (< 1000 devices)

Recommended: Marketplace App (Intune Connect or Microsoft Intune Importer)

Rationale:

- Quick deployment with vendor support
- Predictable licensing costs
- Minimal internal development required
- Regular updates and maintenance included
- Proven solutions with active user base

Implementation Timeline: 1-2 weeks

Scenario 2: Enterprise Organization with JSM Premium/Enterprise

Recommended: Assets Data Manager with Intune Adapter

Rationale:

- No additional cost (included with JSM Premium/Enterprise)
- Native Atlassian solution with ongoing investment
- Scalable to 30+ data sources if needed
- Built-in data quality features (cleansing, deduplication)
- Strategic alignment with Atlassian roadmap

Implementation Timeline: 2-4 weeks (including learning curve)

Scenario 3: Complex Multi-Source Environment

Recommended: xAssets or Custom API Integration

Rationale:

- Need to integrate multiple asset sources (Intune, SCCM, Jamf, Network Discovery)
- Require deep asset intelligence beyond Intune data
- Want comprehensive visibility in JIRA tickets
- Have development resources for custom solution maintenance

Implementation Timeline:

- xAssets: 2-4 weeks
 - Custom API: 4-8 weeks
-

Scenario 4: Budget-Constrained or Pilot Phase

Recommended: Assets Data Manager or Custom API Integration

Rationale:

- Minimize additional costs during pilot/proof-of-concept
- Leverage existing JSM Premium/Enterprise license
- Build internal expertise before committing to additional tools
- Proof of concept before marketplace app purchase

Implementation Timeline: 4-6 weeks

JIRA DC to Cloud Assets Migration**Current State Assessment**

Before migrating, document your JIRA DC Assets configuration:

Schema Documentation:

- ☐ Object schema names and keys
- ☐ Object types and their hierarchies
- ☐ Attributes (name, type, default values)
- ☐ Reference relationships between object types
- ☐ Unique constraints
- ☐ Icons and configuration
- ☐ Import configurations (if Intune sync exists on DC)
- ☐ Automation rules
- ☐ Assets custom fields in JIRA issues
- ☐ User/group attributes and their mappings

Data Inventory:

- ☐ Total number of object schemas
- ☐ Objects per schema (check cloud limits)
- ☐ Asset history requirements
- ☐ Attachments and avatars (not migrated automatically)
- ☐ Connected JIRA issues/tickets

Integration Points:

- ☐ Existing Intune integration method (if any)
 - ☐ AD on-prem sync configuration
 - ☐ Any custom import scripts
 - ☐ Third-party integrations
 - ☐ API consumers
-

Migration Options

Option A: Atlassian Jira Cloud Migration Assistant (Recommended)

Prerequisites:

- JIRA Service Management Data Center 4.15+ with bundled Assets
- JIRA Cloud Migration Assistant 1.12.48+
- JSM Premium or Enterprise plan on Cloud
- Within cloud data limits (check before migration)

Migration Coverage:

-  Object schemas (structure and data)
-  Object types and attributes
-  Objects and their relationships
-  Object labels
-  Object history (up to 100 old/new values per entry)
-  Automation rules (with limitations)
-  Connected JIRA issues (automatic linking)
-  Attachments and avatars
-  Import configurations
-  Active Directory integration specifics

Key Limitations:

1. Maximum 20 objects per custom field association
2. Unique constraints limited to 2 per object type
3. AQL queries may need attribute name sanitization
4. Same schema cannot be migrated twice (delete from cloud to re-migrate)
5. Object keys regenerated (JQL queries need updating)

Migration Strategy:

1. **First Migration Run:** Migrate all object schemas
2. **Subsequent Migrations:** Migrate related projects
 - Object schema linking happens automatically if schemas already in cloud
 - Linking will NOT happen if projects migrated before schemas

Steps:

1. Review data limits: <https://support.atlassian.com/migration/docs/prepare-for-your-migration/>
2. Backup JIRA DC environment
3. Open JIRA Cloud Migration Assistant in DC
4. Choose migration options:
 - Select "All object schemas" or "Only object schemas related to selected projects"
 - Include related users and groups
5. Run migration
6. Verify schema structure in Cloud

7. Verify object data integrity
 8. Update JQL queries with new object keys
 9. Recreate import configurations
 10. Test Assets custom fields in issues
-

Option B: Insight Assets Backup & Migration Apps

Required Apps:

- Source (DC): Insight Assets Cloud Migration Assistant
- Destination (Cloud): Insight Assets Backup & Migration

Capabilities:

- Migrate complete object schemas
- Includes all attribute types (users, groups)
- Preserves values if users/groups exist on destination
- Object history migration supported
- More flexible than native migration tool

Process:

1. Install both apps (source and destination)
2. Set up "Current Connection" in Cloud instance
3. Add Data Center instance in Cloud app
4. Sync schemas from DC
5. Select schema and backup to restore
6. Choose destination and restore
7. Verify migration completeness

Advantages:

- Can migrate same schema multiple times
- More granular control over what to migrate
- Object history included
- Vendor support from Twinit

Disadvantages:

- Additional cost for apps
 - Requires installation and configuration
 - Two-app dependency
-

Option C: Manual CSV Export/Import

When to Use:

- Marketplace apps unavailable
- Schema structure changes needed during migration
- Data cleansing required before import
- Pilot/proof-of-concept scenarios

Process:

1. Export from JIRA DC:

- Navigate to Assets > Object Schema
- For each object type: Bulk Actions (gear icon) > Export Objects > Export
- Downloads CSV files with object data
- Export schema structure separately if needed

2. Prepare Data:

- Update user references from emails to Account IDs:

- a. Export users from Cloud: Admin > Users > Export users
 - b. Use VLOOKUP or script to map email → Account ID
 - c. Replace email references in CSV with Account IDs

- Validate CSV structure matches Cloud schema
- Clean/transform data as needed

3. Recreate Schema in Cloud:

- Manually recreate object schema structure
- Create object types with identical attributes
- Set up relationships between object types
- Configure icons, default values, etc.

4. Import Data:

- Assets > Object Schema > Import > CSV
- Map CSV columns to attributes
- Import objects (may need multiple passes for relationships)
- Verify import completeness

Pros:

- Full control over data transformation
- Opportunity to clean/restructure
- No additional tools required
- No dependency on migration success

Cons:

- Time-consuming and manual
 - Risk of data mapping errors
 - Object keys will not match (impacts JQL)
 - Must recreate schema structure manually
 - Import configurations not preserved
 - Automation rules must be recreated
-

Post-Migration Schema Reconciliation

Critical Tasks:

1. Update Assets Custom Fields:

- Object keys changed during migration
- Existing issue links will be broken
- Workarounds:
 - Use REST API to bulk update issue fields with new object keys
 - Create mapping script: old key → new key
 - Refer to: <https://confluence.atlassian.com/jirakb/workarounds-to-automatically-link-the-assets-object-to-jira-issues>

2. Recreate Import Configurations:

- Import sources are not migrated

- If existing Intune integration on DC, must be rebuilt in Cloud
- Use one of the recommended integration options

3. **Test Automation Rules:**

- Verify object-triggered automation still functions
- Check for broken references to old object keys
- Update AQL queries if needed

4. **Verify Relationships:**

- Cross-schema object references
- Parent-child relationships
- Dependencies and links

5. **User/Group Mapping:**

- Verify all user attributes resolved correctly
 - Check group memberships
 - Test access permissions on schemas
-

Implementation Steps

Phase 1: Planning and Prerequisites (Week 1-2)

Assessment:

1. Inventory current JIRA DC Assets configuration
2. Document Intune environment (tenant, devices, apps)
3. Identify stakeholders and requirements
4. Determine integration option based on scenarios
5. Review cloud data limits and constraints

Prerequisites Checklist:

- ❑ Azure Tenant ID and admin access
- ❑ JIRA Cloud site with JSM Premium or Enterprise
- ❑ Intune devices enrolled and managed
- ❑ Azure App Registration permissions available
- ❑ Budget approval (if marketplace app chosen)
- ❑ Development resources (if custom integration)
- ❑ Migration window scheduled
- ❑ Backup of JIRA DC environment
- ❑ Test environment available (recommended)

Phase 2: Azure Configuration (Week 2)

Azure App Registration (required for all integration options):

1. Create App Registration:

Azure Portal > Microsoft Entra ID > App registrations > New registration

Name: "Intune-JIRA-Assets-Integration"

Supported account types: Single tenant

2. Configure API Permissions:

App > API permissions > Add permission > Microsoft Graph

Required Permissions:

- DeviceManagementManagedDevices.Read.All
- User.Read.All (if syncing user data)
- Application.Read.All (if syncing application data)

Grant admin consent

3. Create Client Secret:

App > Certificates & secrets > New client secret

Description: "Intune JIRA Integration"

Expires: Choose appropriate duration (12-24 months recommended)

IMPORTANT: Copy and securely store the secret value immediately

4. Note Configuration Values:

Application (client) ID: [copy from Overview]
Directory (tenant) ID: [copy from Overview]
Client secret value: [copied in step 3]

Phase 3: JIRA Cloud Assets Preparation (Week 2-3)

If Migration Not Yet Complete:

1. Execute JIRA DC to Cloud migration following Option A or B
2. Verify schema migration completeness
3. Complete post-migration reconciliation tasks

Schema Configuration:

1. Review/Create Object Schema:

Name: "IT Assets" or similar
Key: ITASSET (or existing key from DC)

2. Verify/Create Object Types:

Minimum required for Intune integration:

- Computer / Laptop / Workstation
- Operating System
- User / Employee
- Manufacturer
- Model
- Discovered Application (optional)

3. Configure Attributes:

Computer Object Type:

- Name (Default text)
- Serial Number (text, unique)
- Manufacturer (object reference)
- Model (object reference)
- Operating System (object reference)
- Primary User (object reference or user)
- Device ID (text) - Intune device ID
- Last Sync Date (date/time)
- Enrollment Date (date)
- Compliance Status (text or status)
- Management State (text)
- MAC Address (text)
- IP Address (text)
- Free Storage (number)
- Total Storage (number)

Operating System Object Type:

- Name (text)
- Version (text)
- Platform (text) - Windows, macOS, iOS, Android

User Object Type:

- Display Name (text)
- Email (text, unique)
- UPN (text, unique)
- Entra ID (text) - Account ID
- Department (text)
- Manager (object reference or user)

Phase 4: Integration Implementation (Week 3-4)

If Using Marketplace App:

Intune Connect for JSM Assets:

1. Install app from Marketplace
2. Configure app settings:
 - Enter Tenant ID, Client ID, Client Secret
 - Configure sync schedule (hourly, daily, weekly)
 - Map Intune fields to Assets attributes

- Set filtering rules (if needed)
3. Run initial sync
 4. Verify data import
 5. Review and adjust field mappings
 6. Enable automated synchronization

Microsoft Intune Importer:

1. Install app from Marketplace
 2. Follow app configuration wizard
 3. Provide Azure credentials
 4. Map data fields to object schema
 5. Execute initial import
 6. Validate object creation
 7. Schedule recurring imports
-

If Using Assets Data Manager:

1. **Access Data Manager:**

JSM > Assets > Data Manager (top navigation)

2. **Configure Intune Adapter:**

Adapters > Add New Adapter > Microsoft Intune

Name: "Intune Device Sync"

Object Class: "Computer" (or your device object type)

Data Source Name: "Microsoft Intune"

Data Source Type: "CMDB"

Intune Configuration:

- Tenant-ID: [from Azure]
- Client-ID: [from Azure]
- Client Secret: [from Azure]
- API Timeout: 9000 (recommended)
- API Limit: 0 (no limit) or specific value

3. Configure Additional Adapters (optional):

- Entra ID User Adapter (for user data)
- Entra ID Device Adapter (for Azure AD devices)

4. Test Connection:

Test API connection

Review sample data returned

Verify field mappings

5. Set Up Data Transformation:

Data Manager > Transformations

Common transformations:

- Normalize manufacturer names
- Deduplicate devices
- Map OS names to standard values
- Calculate asset age
- Set location based on naming convention

6. Create Import Job:

Configure schedule (daily at off-peak hours recommended)
Set incremental vs full sync
Configure error handling
Set notification preferences

7. Execute Initial Import:

Run manual import
Monitor progress
Review raw data in Data Manager
Check for import errors

8. Verify in Assets:

Navigate to Assets > Object Schema
Verify objects created
Check attribute population
Review relationships
Validate data quality

If Using Custom API Integration:

Sample PowerShell Script Structure:

```
powershell
```

Configuration

```
$tenantId = "your-tenant-id"
$clientId = "your-client-id"
$clientSecret = "your-client-secret"
$jiraCloudId = "your-cloud-id"
$jiraWorkspaceId = "your-workspace-id"
$jiraApiToken = "your-jira-token"
$jiraEmail = "your-email@company.com"
```

Get Microsoft Graph Token

```
$tokenBody = @{
    Grant_Type = "client_credentials"
    Scope      = "https://graph.microsoft.com/.default"
    Client_Id  = $clientId
    Client_Secret = $clientSecret
}
$tokenResponse = Invoke-RestMethod -Uri "https://login.microsoftonline.com/$tenantId/oauth2/v2.0/token" -Method POST
$msGraphToken = $tokenResponse.access_token
```

Get Intune Devices

```
$intuneDevices = Invoke-RestMethod -Uri "https://graph.microsoft.com/v1.0/deviceManagement/managedDevices" `
    -Headers @{ Authorization = "Bearer $msGraphToken" } -Method GET
```

Transform and Upload to JIRA Assets

```
foreach ($device in $intuneDevices.value) {
    $objectData = @{
        objectType = "123" # Your Computer object type ID
        attributes = @(
            @{ objectTypeAttributeId = "456"; objectAttributeValues = @( @{ value = $device.deviceName } ) }
            @{ objectTypeAttributeId = "457"; objectAttributeValues = @( @{ value = $device.serialNumber } ) }
            # Add more attributes...
        )
    }
}
```

Create/Update in JIRA Assets

```
$jiraHeaders = @{
    "Authorization" = "Basic " + [Convert]::ToBase64String([Text.Encoding]::ASCII.GetBytes("${jiraEmail}:${jiraApiToken}"))
    "Content-Type" = "application/json"
}
```

```
$jiraUri = "https://api.atlassian.com/ex/jira/$jiraCloudId/jsm/assets/workspace/$jiraWorkspaceId/v1/object/create"
```

```
Invoke-RestMethod -Uri $JiraUri -Method POST -Headers $JiraHeaders -Body ($ObjectData | ConvertTo-Json -Depth 10)
}
```

Implementation Steps:

1. Develop and test script in development environment
 2. Implement error handling and logging
 3. Configure secure credential storage (Azure Key Vault recommended)
 4. Set up scheduling (Azure Automation, Task Scheduler, or cron)
 5. Deploy to production environment
 6. Monitor initial syncs
 7. Establish alerting for failures
 8. Document maintenance procedures
-

Phase 5: Testing and Validation (Week 4-5)

Data Validation:

Test Cases:

- ☐ Device count matches Intune inventory
- ☐ All device attributes populated correctly
- ☐ User assignments accurate
- ☐ Operating system data correct
- ☐ Manufacturer and model information valid
- ☐ Serial numbers unique and accurate
- ☐ Last sync timestamp updating
- ☐ Incremental sync only updates changed devices
- ☐ Deleted devices removed from Assets
- ☐ New devices appear after enrollment
- ☐ Compliance status reflects Intune accurately
- ☐ Application inventory (if configured) complete

Integration Testing:

- ❑ Sync runs on schedule without errors
- ❑ Error notifications working
- ❑ API authentication refreshes correctly
- ❑ Data transformation rules working
- ❑ Deduplication preventing duplicates
- ❑ Performance acceptable for device count
- ❑ JIRA Assets queries returning expected results
- ❑ Assets fields in JIRA issues linking correctly
- ❑ Reporting and dashboards displaying data
- ❑ Automation rules triggered appropriately

User Acceptance Testing:

- ❑ Service desk can find devices by serial number
- ❑ User-to-device relationships visible
- ❑ Device history tracked correctly
- ❑ Search performance acceptable
- ❑ Mobile app displays device information
- ❑ Relevant attributes available for workflows
- ❑ Reporting requirements met

Phase 6: Go-Live and Optimization (Week 5-6)

Cutover Steps:

1. Verify sync running successfully for 1 week
2. Train service desk on new Assets capabilities
3. Update documentation and procedures
4. Communicate changes to stakeholders
5. Monitor performance and adjust as needed
6. Gather user feedback
7. Optimize queries and reports

Ongoing Maintenance:

- ❑ Monitor sync job success rate
 - ❑ Review data quality metrics
 - ❑ Update field mappings as Intune changes
 - ❑ Refresh Azure credentials before expiry
 - ❑ Audit unused attributes
 - ❑ Optimize performance as data grows
 - ❑ Regular backup of Assets schemas
 - ❑ Review and update automation rules
 - ❑ Keep integration apps updated
-

Known Limitations and Challenges

Atlassian Platform Limitations

1. Assets Data Limits (Cloud):

- Must stay within cloud platform limits
- Varies by JSM plan (Premium vs Enterprise)
- Check current limits: <https://support.atlassian.com/jira-service-management-cloud/docs/what-are-the-limits-of-assets/>

2. Migration Constraints:

- Object keys change during migration (impacts JQL queries)
- Maximum 20 objects per custom field association
- Only 2 unique constraints per object type allowed
- Attachments and avatars not migrated
- Import configurations must be recreated
- AQL queries may need attribute sanitization

3. Assets API Limitations:

- Rate limiting applies to API calls
- Bulk operations have size limits
- Authentication token expiration handling required

4. Data Manager Constraints:

- No generic REST API adapter (as of April 2025)
- Learning curve for transformation rules
- Limited documentation for complex scenarios

- Beta features may have stability issues
-

Intune Integration Challenges

1. Incomplete Data:

- Intune may not capture all software installations
- Local admin users can install software invisible to Intune
- Browser extensions and user-level apps may be missed
- Network-based assets not visible to Intune
- Shadow IT remains undetected

2. Data Freshness:

- Device check-in frequency affects data recency
- Offline devices show stale information
- Real-time compliance status may lag
- Sync schedules create data latency

3. Authentication and Permissions:

- Azure app registration requires Global Administrator or Intune Admin
- Client secret expiration requires renewal
- Permission changes may break integration
- MFA can complicate service account authentication

4. Field Mapping:

- Intune and Assets schemas may not align perfectly
 - Custom attributes require manual mapping
 - Data transformation logic needed for some fields
 - Manufacturer name variations cause duplicates
-

Marketplace App Considerations

1. Vendor Dependency:

- Reliant on vendor for bug fixes and updates
- App discontinuation risk
- Support response times vary

- Feature requests depend on vendor roadmap

2. Cost Management:

- Subscription costs add up over time
- Price increases possible
- May need multiple apps for full functionality
- License tier considerations

3. Upgrade Impacts:

- Atlassian platform updates may break apps
 - App updates may change behavior
 - Testing needed before upgrades
 - Compatibility matrix to track
-

Custom Integration Challenges

1. Development and Maintenance:

- Requires programming expertise
- Ongoing maintenance burden
- No vendor support for custom code
- Documentation must be created internally
- Knowledge transfer risk if developer leaves

2. API Changes:

- Microsoft Graph API evolves over time
- Atlassian Assets API updates may require changes
- Authentication methods can be deprecated
- Beta endpoints may become stable or removed

3. Security:

- Credential management critical
- Secure storage required for secrets
- Audit logging needed
- Compliance considerations

4. Scaling:

- Performance testing required

- Batch processing for large datasets
 - Rate limit handling
 - Error recovery mechanisms
-

Migration-Specific Challenges

1. Data Fidelity:

- Risk of data loss during migration
- Relationship integrity must be verified
- User attribute mapping complexities
- Historical data may be truncated

2. Cutover Timing:

- Service interruption during migration
- Parallel run complexity if needed
- Rollback difficulty once committed
- User training timing

3. Schema Differences:

- DC schema flexibility vs Cloud constraints
 - Custom importers not available in Cloud
 - Automation rule differences
 - UI/UX changes require adjustment
-

Known Issues

1. Mobile App Limitation (ID-6578):

- JIRA Cloud mobile app not compatible with Microsoft Intune Conditional Access
- Managed browser requirements conflict with app's browser
- Workaround: Use mobile web browser instead of app
- Affects users with strict Conditional Access policies

2. Secure Admin Session (JIRA DC):

- Websudo doesn't redirect to IdP for SSO
- Re-authentication uses local credentials only

- Feature request: JRASERVER-69311
- Workaround: Disable secure admin sessions (security impact)

3. Asset Panel Deprecation:

- Atlassian deprecated native Asset Panel software (December 2024)
 - APIs turned off January 2025
 - Affects custom integrations using old panel
 - Replacement: Third-party solutions like xAssets or custom implementations
-

Vendor Recommendations

Atlassian's Official Stance

As of February 2026, Atlassian:

-  Acknowledges Intune integration as important feature
-  Supports Marketplace app development for import integrations
-  Provides Data Manager with Intune adapter
-  Offers REST API for custom integrations
-  Does not provide out-of-box Intune integration
-  Has not committed to native integration timeline
-  Working on enabling Marketplace ecosystem for integrations

Official Resources:

- Assets Import API: <https://developer.atlassian.com/cloud/insight/imports/introduction/>
 - Assets REST API Guide: <https://developer.atlassian.com/cloud/assets/assets-rest-api-guide/>
 - Data Manager Documentation: <https://support.atlassian.com/assets/docs/data-manager-assets-data-manager/>
-

Recommended Path by Use Case

For Organizations Migrating from DC to Cloud:

1. Complete Assets schema migration using Jira Cloud Migration Assistant
2. Validate schema and data integrity post-migration
3. Implement one of the integration options based on:

- Budget availability → Marketplace app vs Data Manager
- Technical resources → Custom integration if dev team available
- Complexity → Multi-source needs may require xAssets or custom solution

4. Recreate automation rules and workflows in Cloud

5. Update documentation and train users

For New JIRA Cloud Deployments:

1. Start with Assets Data Manager (included with JSM Premium/Enterprise)
2. Configure Intune adapter and test
3. If Data Manager doesn't meet needs, evaluate marketplace apps
4. Build custom integration only if specific requirements demand it

Best Practices Recommended by Community:

- Start with simpler solution and evolve as needed
 - Prioritize vendor-supported solutions over custom code
 - Plan for ongoing maintenance regardless of approach
 - Test thoroughly before production deployment
 - Document integration configuration and troubleshooting
 - Maintain schema documentation for future migrations
 - Regular review of data quality and sync success
-

Required Infrastructure and Prerequisites

Azure Requirements

Minimum:

- Azure tenant with Microsoft Intune license
- Devices enrolled in Intune
- Global Administrator or Intune Service Administrator access
- Ability to create app registrations

Recommended:

- Azure Key Vault for secret storage (if custom integration)

- Azure Automation account (if using PowerShell sync scripts)
- Azure Monitor for logging and alerting
- Dedicated service account for integration
- Non-expiring or long-lived client secret

API Permissions:

Microsoft Graph API:

- DeviceManagementManagedDevices.Read.All (Application)
 - User.Read.All (Application) - if syncing user data
 - DeviceManagementApps.Read.All (Application) - if syncing apps
 - Directory.Read.All (Delegated) - for enhanced user details
-

JIRA Cloud Requirements

Minimum:

- JIRA Service Management Cloud instance
- Premium or Enterprise plan (Assets included)
- Site admin access for configuration
- Assets schema created and configured

Recommended:

- Test/sandbox environment for validation
- API token created for automation
- User with Assets admin permissions
- Documented schema structure
- Backup procedures in place

Plan Considerations:

- **Standard Plan:** Assets not included (requires upgrade)
 - **Premium Plan:** Assets + Data Manager included
 - **Enterprise Plan:** Assets + Data Manager + advanced features
-

Network and Security

Firewall Rules:

Outbound Access Required (if using Azure VM/server for custom integration):

- api.atlassian.com (443)
- graph.microsoft.com (443)
- login.microsoftonline.com (443)
- your-jira-instance.atlassian.net (443)

Security Considerations:

- Secure storage for client secrets and API tokens
 - Principle of least privilege for service accounts
 - Regular credential rotation
 - Audit logging enabled
 - MFA for admin accounts
 - Review app permissions regularly
-

Skills and Resources

Technical Skills Needed:

For Marketplace Apps:

- Basic Azure admin (app registration)
- JIRA Assets configuration
- Schema design understanding

For Assets Data Manager:

- Azure admin (app registration)
- Data transformation concepts
- JIRA Assets schema design
- Basic troubleshooting

For Custom Integration:

- Programming (PowerShell, Python, or similar)
- REST API experience
- Microsoft Graph API knowledge

- JIRA Assets API familiarity
- Error handling and logging
- Authentication flow understanding
- Scheduling/automation platforms

Team Roles:

- Project Manager: coordinate migration and implementation
 - Azure Administrator: app registration and permissions
 - JIRA Administrator: Assets configuration and schema design
 - Developer: custom integration development (if needed)
 - Service Desk Lead: requirements gathering and UAT
 - IT Security: security review and approval
-

Budget Considerations

One-Time Costs:

- Marketplace app licenses (if selected)
- Migration tool licenses (if using third-party)
- Consulting/implementation services (optional)
- Training for staff

Recurring Costs:

- Marketplace app subscription (monthly/annual)
- Azure VM/hosting (if custom integration)
- Data Manager (included with JSM Premium/Enterprise)
- Maintenance and support

Hidden Costs:

- Developer time for custom solutions
- Ongoing maintenance and monitoring
- Schema redesign if migration goes poorly
- Training and documentation
- Testing environments

Post-Migration Considerations

Optimization Opportunities

1. Data Quality Improvement:

- Establish data governance policies
- Regular data cleansing schedules
- Standardize naming conventions
- Implement validation rules
- Deduplicate existing objects

2. Automation Enhancement:

- Automated ticket creation for device issues
- Proactive alerting for compliance violations
- Auto-assignment based on device location
- Workflow triggers for device lifecycle events
- Integration with change management

3. Reporting and Dashboards:

- Device inventory reports
- Compliance status dashboards
- Software license utilization
- Hardware refresh planning
- Cost center allocations

4. Advanced Use Cases:

- CMDB relationships (devices → applications → services)
- Impact analysis for incidents
- Change risk assessment based on affected devices
- SLA management by device type
- Capacity planning and forecasting

Continuous Improvement

Quarterly Reviews:

- Sync success rate and data quality metrics
- User satisfaction with Assets data
- Performance of queries and reports
- Schema optimization opportunities
- Integration reliability

Annual Activities:

- Renew Azure client secrets (before expiry)
 - Review and optimize object schema
 - Audit unused attributes and object types
 - Evaluate new Atlassian features
 - Consider integration platform upgrades
 - Assess ROI of current solution
-

Scaling Considerations

As Device Count Grows:

- Monitor API rate limits
- Optimize sync schedules
- Implement incremental sync vs full sync
- Consider batch processing strategies
- Review Data Manager performance
- Evaluate need for caching layer

As Integrations Expand:

- Document integration architecture
 - Standardize data mapping approach
 - Consider xAssets for multi-source consolidation
 - Implement integration monitoring dashboard
 - Plan for schema expansion
 - Maintain integration dependency matrix
-

Risk Management

Integration Failure Scenarios:

Risk: Azure client secret expires

Mitigation: Calendar reminders 30/60 days before expiry

Automated alerting on auth failures

Documented renewal procedure

Risk: Marketplace app discontinued

Mitigation: Export data regularly

Plan for alternative integration

Monitor vendor communications

Risk: API rate limiting

Mitigation: Implement exponential backoff

Schedule syncs during off-peak hours

Cache frequently accessed data

Risk: Data quality degradation

Mitigation: Regular data quality audits

Automated validation rules

User feedback mechanisms

Risk: Schema changes break integration

Mitigation: Test schema changes in sandbox

Version control for transformations

Change management process

Appendix

Useful Links

Atlassian Resources:

- JIRA Cloud Migration Guide: <https://support.atlassian.com/migration/>
- Assets Documentation: <https://support.atlassian.com/jira-service-management-cloud/docs/what-is-assets/>
- Data Manager Guide: <https://support.atlassian.com/assets/docs/data-manager-assets-data-manager/>
- Assets REST API: <https://developer.atlassian.com/cloud/assets/>
- Import API: <https://developer.atlassian.com/cloud/insight/imports/introduction/>

Microsoft Resources:

- Microsoft Graph API: <https://docs.microsoft.com/en-us/graph/>
- Intune API Reference: <https://docs.microsoft.com/en-us/graph/api/resources/intune-graph-overview>
- Azure App Registration: <https://docs.microsoft.com/en-us/azure/active-directory/develop/>

Community Resources:

- Atlassian Community: <https://community.atlassian.com/>
 - Feature Request (Intune Integration): <https://jira.atlassian.com/browse/JSDCLOUD-10863>
 - Assets Migration KB: <https://confluence.atlassian.com/jirakb/migrate-assets-data-from-data-center-into-assets-in-jsm-cloud>
-

Glossary

- **Assets:** JIRA Service Management's CMDB and asset management module (formerly Insight)
 - **Object Schema:** The database structure containing object types and their relationships
 - **Object Type:** A category of assets (e.g., Computer, User, Application)
 - **Attribute:** A field within an object type (e.g., Serial Number, OS Version)
 - **Data Manager:** Atlassian's data integration and quality management platform
 - **Adapter:** Pre-built connector in Data Manager for specific data sources
 - **AQL:** Assets Query Language for searching and filtering objects
 - **Intune:** Microsoft's cloud-based endpoint management service
 - **Azure Entra ID:** Microsoft's identity and access management service (formerly Azure AD)
 - **CMDB:** Configuration Management Database
 - **JSM:** JIRA Service Management
 - **SSO:** Single Sign-On
 - **SAML:** Security Assertion Markup Language (authentication protocol)
-

Contact and Support

For Atlassian Support:

- JIRA Cloud: <https://support.atlassian.com/>
- Migration Assistance: support@atlassian.com

- Community Forums: <https://community.atlassian.com/>

For Microsoft Support:

- Microsoft Graph API: <https://developer.microsoft.com/en-us/graph/support>
- Intune Support: <https://docs.microsoft.com/en-us/mem/get-support>

For Marketplace Apps:

- Check app listing for vendor support contact
- Usually: support email provided on Marketplace page
- Response times vary by vendor

Document Change Log

Version	Date	Author	Changes
1.0	Feb 2026	Initial	Initial comprehensive guide created

End of Document